

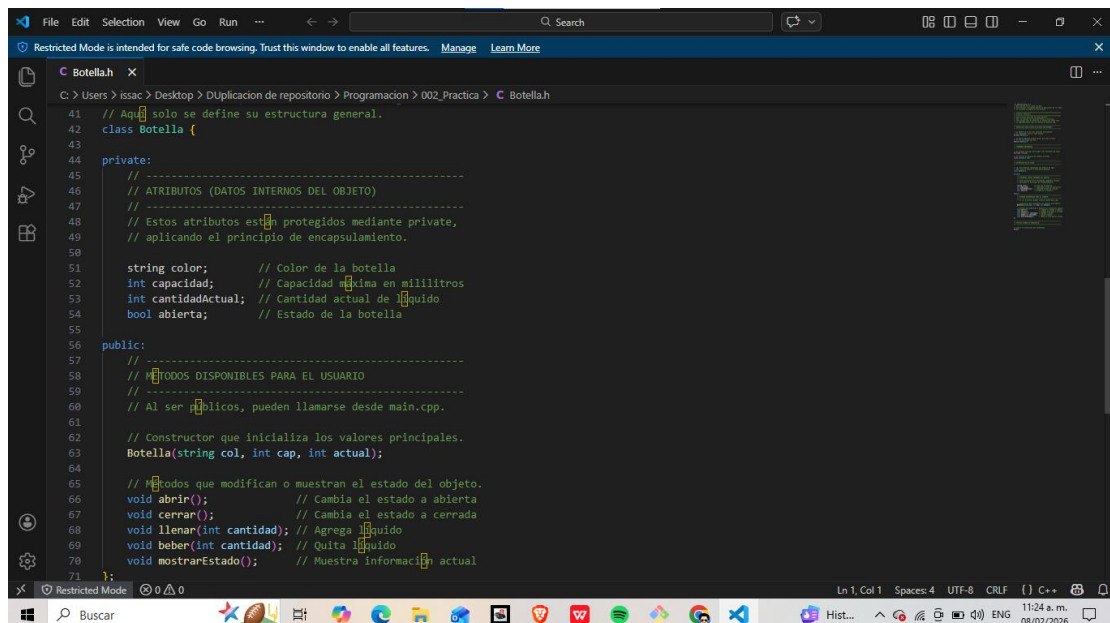
Cómo se realizó la práctica

— Práctica 02 POO

Para realizar esta práctica primero elegí un objeto del mundo real que fuera fácil de representar en un programa. En este caso seleccioné una **botella de agua**, ya que es un objeto cotidiano y permite aplicar reglas claras como estar abierta o cerrada, tener una capacidad máxima y contener cierta cantidad de líquido.

Después identifiqué sus características principales, que se convirtieron en los atributos de la clase: color, capacidad, cantidadActual y el estado de abierta o cerrada.

También analicé qué acciones podía realizar una botella y las convertí en métodos, como abrir, cerrar, llenar, beber y mostrarEstado.



```
1  // Aqu solo se define su estructura general.
2  class Botella {
3
4  private:
5      // -----
6      // ATRIBUTOS (DATOS INTERNOS DEL OBJETO)
7      // -----
8      // Estos atributos están protegidos mediante private,
9      // aplicando el principio de encapsulamiento.
10
11     string color;        // Color de la botella
12     int capacidad;       // Capacidad máxima en mililitros
13     int cantidadActual;  // Cantidad actual de líquido
14     bool abierta;        // Estado de la botella
15
16 public:
17     // -----
18     // MÉTODOS DISPONIBLES PARA EL USUARIO
19     // -----
20     // Al ser públicos, pueden llamarse desde main.cpp.
21
22     // Constructor que inicializa los valores principales.
23     Botella(string col, int cap, int actual);
24
25     // Métodos que modifican o muestran el estado del objeto.
26     void abrir();         // Cambia el estado a abierta
27     void cerrar();        // Cambia el estado a cerrada
28     void llenar(int cantidad); // Agrega líquido
29     void beber(int cantidad); // Quita líquido
30     void mostrarEstado(); // Muestra información actual
31
32 }
```

Una vez definido esto, separé el proyecto en tres archivos para aplicar modularidad:

- Botella.h, donde declaré la clase, los atributos privados y los métodos públicos sin escribir la lógica.
- Botella.cpp, donde implementé los métodos y agregué las validaciones necesarias para proteger la información interna, como no permitir cantidades negativas, no exceder la capacidad y no realizar acciones si la botella está cerrada.
- main.cpp, donde creé dos objetos distintos de la misma clase y probé su funcionamiento llamando a los métodos.

En la implementación utilicé **encapsulamiento**, colocando los atributos como privados para que no pudieran modificarse directamente desde el programa principal. De esta forma, cualquier cambio tenía que pasar por los métodos públicos, donde se revisan las reglas.

Con esta práctica pude aplicar los conceptos de abstracción, al modelar solo lo más importante del objeto real; encapsulamiento, al proteger los datos internos; y modularidad, al separar el código en distintos archivos.